

Detailed Project Log: Side-Channel Detection via HPC AI Analysis

1. Executive Summary & Intent

This document serves as a comprehensive handover of the project's current state, methodologies, and infrastructure. The objective is to detect **Cache-based Side-Channel Attacks (SCAs)** using **Hardware Performance Counters (HPCs)**. We have successfully moved from raw hardware telemetry collection to a high-accuracy ML detection model (99.98% Accuracy).

2. Infrastructure & Data Collection Environment

The project was executed across two distinct environments:

A. The Collection Server (Bare Metal / Controlled Node)

- **OS:** Ubuntu Linux (6.17.0 kernel-generic).
- **Hardware Context:** Access to physical HPC registers via the perf subsystem.
- **Monitoring Tool:** linux-tools-generic (perf stat).
- **Sampling Rate:** 50ms intervals (-I 50).
- **Hardware Events Tracked:**
 1. cache-misses
 2. cache-references
 3. instructions
 4. cycles
 5. branches
 6. branch-misses

B. The Training Environment (AWS EC2)

- **Instance Type:** t3.large (2 vCPU, 8GB RAM).
 - **OS:** Ubuntu 24.04.4 LTS (Noble Numbat).
 - **Setup:** Python 3.12 Virtual Environment (venv), Jupyter Notebook (served on port 8888, accessed via SSH tunnel).
-

3. Data Collection Methodology (Phase 1 & 2)

The Benign Baseline (`benign_data.csv`)

- **Victim Process:** A persistent benchmarking loop simulating normal computational workload.
- **Collection Strategy:** Single-stream perf collection for **1 hour**.
- **Integrity Control:** Resolved a "Double Collector" bug where multiple PID's were writing to the same CSV, causing duplicate timestamps.
- **Final Result:** 73,006 clean samples.

The Attack Signature (`attack_data_final.csv`)

- **Attacker Process:** A custom C-binary (`./attacker`) implementing high-frequency cache eviction.
 - **Interference Pattern:** Prime+Probe style behavior designed to cause L3 cache contention.
 - **Collection Strategy:** 1-hour run concurrent with the victim process.
 - **Visual Verification:** Observed sharp logarithmic spikes in cache-misses (reaching ~1.4M events) compared to the stable benign average (~128k).
-

4. Feature Engineering & Preprocessing (Phase 3)

Data Transformation Pipeline

1. **Vertical to Horizontal Pivot:** The raw perf output is "long" (1 event per row). We used a `pivot_table` indexed by `timestamp` to create a single row containing all 6 hardware sensors.
 2. **Ground Truth Labeling:**
 - Label 0: Benign telemetry.
 - Label 1: Attack telemetry.
 3. **Cleaning:** Cast all values to numeric and dropped initialization noise (NaNs).
 4. **Final Matrix:** Combined dataset resulted in **150,519 total samples**.
-

5. Model Architecture & Evaluation (Phase 4)

The Classifier: Random Forest (Ensemble)

We chose a Random Forest due to its resilience against noise and ability to handle non-linear relationships in HPC bursts.

- **Configuration:** 100 estimators, max depth of 15.
- **Accuracy:** 99.98%.
- **Critical Features (Ranked):**
 1. cache-references (Highest predictive power)
 2. cache-misses
 3. instructions
 4. branch-misses

6. Project Achievement Status (Gap Mapping)

Mapped against **17.pdf** Research Objectives:

Goal	Status	Technical Result
OBJ 1: Feature Extraction	Done	Captured 6 primary HPC events at 50ms granularity.
OBJ 2: Dataset Generation	Done	150k+ labeled samples for Flush+Reload/Prime+Probe styles.
OBJ 3: ML Ensemble	Done	Random Forest achieving near-perfect separation.
OBJ 4: Cross-Arch Eval	To-Do	Requires testing on Intel vs AMD vs ARM hardware.
OBJ 5: Real-Time Module	To-Do	Transitioning from CSV analysis to live inference.

7. Operational Instructions for the Next Phase

To continue from this point, the following assets are required: 1. `benign_data.csv` & `attack_data_final.csv` (Source Data). 2. `RF_model.ipynb` (Current Analysis Pipeline). 3. The `./attacker` and `./victim` binaries (for new architecture tests).

Immediate Next Task: Implement a "Live Monitor" script in Python that uses subprocess to tail perf and pass chunks to the saved .pkl model for real-time threat detection.